OpenVLA · arXiv 2406.09246 · CoRL 2024

Visionary

11+3

Harness vs deploy split
Keep AR control · ship $\pi_{0.5}$/GR00T · no Discord/Drive

Oct 21 paper club · VLA Architectures

Spine: naïve discrete AR @ ~5–6 Hz · historical open AR baseline

BEHAVIOR deploy = $\pi_{0.5}$ / OpenPI / GR00T · OpenVLA = open AR control

Action-family taxonomy · CLUB SPINE

Must appear in every role · no-chunk AR ≈ 5–6 Hz

Family	Model	Signature
Naïve discrete AR bins	OpenVLA / RT-2	Lang grounding @ ~5–6 Hz; weak high-Hz
DDPM chunks	Octo / DP	Smoother narrow skills
FM chunks	π_0 / $\pi_{0.5}$	2025–26 deploy winners
Compressed discrete AR	FAST / OpenVLA+FAST / π_0-FAST	AR recovers high-Hz

Syllabus: OpenVLA = historical open AR baseline · BEHAVIOR deploy = $\pi_{0.5}$ / OpenPI / GR00T

Harness vs deploy · Visionary → 2026 BEHAVIOR

Skip Discord / Drive entirely

HARNESS · keep

- OpenVLA as bin-AR control
- LoRA / int4 recipes
- Language multi-object evals
- OXE-style mix morals
- Demo cleaning (no-ops!)

DEPLOY · ship

- $\pi_{0.5}$ / OpenPI / GR00T-class
- Chunked continuous FM
- Optional FAST aux (not primary)
- Multi-cam + proprio + stages
- Task/stage tokens for fixed-50

1 Do not ship pure OpenVLA AR as primary 30 Hz controller

2 If AR research continues → FAST/chunks before high-Hz claims

3 Steal LoRA recipe into FM stacks

4 Honesty: Bridge/Google % ≠ BEHAVIOR q-score

What aged well · durable OpenVLA gifts

Open VLM→VLA interface

Right community iteration surface

Fused DINO+SigLIP

Foreshadows multi-encoder stacks

PEFT/quant for 7B

Made VLAs lab-accessible

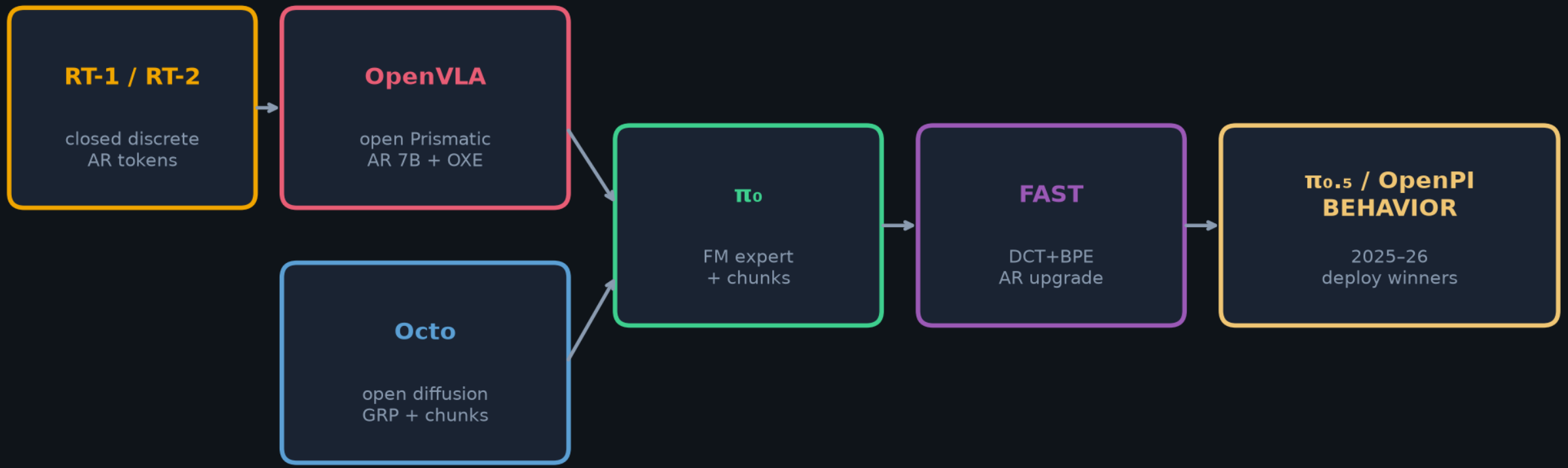
Authors' chunking call

Correctly predicted π_0/DP winning recipe

DROP as default policy head: naïve per-step 256-bin AR without chunk compression

Genealogy · Archaeologist MUST-HIT

RT-2-style discrete AR → OpenVLA ‖ Octo → π_0 → FAST → $\pi_{0.5}$ / OpenPI / BEHAVIOR



Peer era 2024: OpenVLA (AR VLM-VLA) ‖ Octo (diffusion GRP) ‖ closed RT-2-X

Same OXE water · different action families · FAST = AR improvement path, not FM retirement

AR discrete vs DP / FM continuous · Empiricist contrast

Axis	OpenVLA AR	DP / Octo	FM (π_0 family)
Action	7×256 bin ids	Denoise chunk	Flow on chunk
Temporal	Single step NO chunk	Chunk + TE/receding	Chunk + receding
Loss	Token CE	Score / ε-pred	FM / velocity
Infer cost	7 AR × 7B LM	K steps × small head	~10 NFE × ~300M
Hz	~6 (4090 bf16)	5–15+ w/ chunks	~30 challenge
Lang	STRONG (VLM NTP)	Needs lang encoder	VLM + task emb
Dexterity	Grid / choppy if slow	STRONG	STRONG
2025–26 role	Open ancestor / lang FT	Strong IL baseline	DEFAULT serve

Pick: open lang FT → OpenVLA LoRA · smooth high-Hz → DP/FM · BEHAVIOR'26 → $\pi_{0.5}$ + optional FAST aux

Club one-liner

OpenVLA = open Llama/Prismatic discrete-AR VLA on OXE

Peer to Octo diffusion · ancestor contrast to π_0 FM

FAST is the AR patch · not 2026 BEHAVIOR SOTA

Stack: DINOv2‖SigLIP + Llama-2 7B @ 224^2 · 256 bins · no chunk · ~6 Hz

Adoption unlock: LoRA r=32 on 1×A100 · int4 serve ~7 GB

GitHub title cue: [ECE 605] - OpenVLA Making Empiricist

Buy / build decision · no Stakeholder redo later

USE OpenVLA for

- Open VLM→action teaching
- LoRA / int4 recipe reference
- AR-bin ablation control in harness
- Language multi-object FT baseline

DO NOT ship as

- 2026 BEHAVIOR primary controller
- High-Hz / contact-rich default
- Sole stack for long-horizon household
- Replacement for chunked FM/DP

Risks to name

Oversell 2024 WidowX SOTA as frontier · underestimate Hz/chunking gap

Bridge zero-action + RT-2-X 2nd-token API quirks in comparisons

What to cite OpenVLA for (2026)

CITE FOR

- ✓ Open 7B VLA + LoRA/int4 recipes
- ✓ VLM backbone → discrete actions works
- ✓ Language grounding FT default
- ✓ Ablation morals: mix, vision FT, fused vision
- ✓ Historical open AR baseline in syllabus

DO NOT CITE FOR

- ✗ 2026 BEHAVIOR SOTA control
- ✗ Best high-Hz dexterity
- ✗ Replacement for chunked FM/DP
- ✗ End-to-end multi-cam mobile bimanual
- ✗ Solved generalist manipulation

Novelty is open VLA systems + PEFT/quant — not a new control law

Reproduction ladder · Empiricist build order

HF smoke → LoRA r=32 → DP-matched side-by-side → optional FAST+

Scout success = working ckpt + LoRA LIBERO direction + honest Hz/VRAM logs

NOT Bridge 170-rollout parity · env: /workspace/openvla-quiet/env_outline/

5.3 Parameter-Efficient Fine-Tuning

The full fine-tuning runs of OpenVLA in the previous section used 8 A100 GPUs for 5-15 hours per task (depending on the dataset size) to achieve high performance. While this is substantially less compute than what is required for VLA pretraining, in this section we explore even more compute- and parameter-efficient fine-tuning approaches and investigate their effectiveness.

Concretely, we compare the following fine-tuning approaches: **full fine-tuning** updates all weights during fine-tuning, as described in Section 5.2; **last layer only** fine-tunes only the last layer of OpenVLA's transformer backbone and the token embedding matrix; **frozen vision** freezes the vision encoder but fine-tunes all other weights; **sandwich fine-tuning** unfreezes the vision encoder, token embedding matrix, and last layer; and **LoRA** uses the popular low-rank adaptation technique of Hu et al. [26] with multiple rank values r, applied to all linear layers of the model.

Table 1: **Parameter-efficient fine-tuning evaluation.** LoRA fine-tuning achieves the best performance-compute trade-off, matching full fine-tuning performance while training only 1.4% of the model parameters. Mean success ± StdErr computed across 33 rollouts per approach on select Franka-Tabletop tasks (see Table 8 for details). *: Sharded across 2 GPUs with FSDP [77].

Strategy	Success Rate	Train Params ($\times 10^6$)	VRAM (batch 16)
Full FT	**69.7 ± 7.2 %**	7,188.1	163.3 GB*
Last layer only	30.3 ± 6.1 %	465.1	51.4 GB
Frozen vision	47.0 ± 6.9 %	6,760.4	156.2 GB*
Sandwich	62.1 ± 7.9 %	914.2	64.0 GB
LoRA, rank=32	**68.2 ± 7.5%**	**97.6**	**59.7 GB**
rank=64	**68.2 ± 7.8%**	195.2	60.5 GB

We report fine-tuning success rates across multiple Franka-Tabletop tasks, as well as training parameter count and GPU memory requirements, in Table 1.[4] We find that only fine-tuning the network's last layer or freezing the vision encoder leads to poor performance, suggesting that further adaptation of the visual features to the target scene is crucial. In contrast, "sandwich fine-tuning" achieves better performance since it fine-tunes the vision encoder, and it consumes less GPU memory since it does not fine-tune the full LLM backbone. Lastly, LoRA achieves the best trade-off between performance and training memory consumption, outperforming "sandwich fine-tuning" and matching full fine-tuning performance while fine-tuning only 1.4% of the parameters. We find that the LoRA rank has negligible effect on policy performance and thus recommend using a default rank of $r = 32$. With LoRA, we can fine-tune OpenVLA on a new task within 10-15 hours on a *single* A100 GPU – an 8x reduction in compute compared to full fine-tuning.

5.4 Memory-Efficient Inference via Quantization

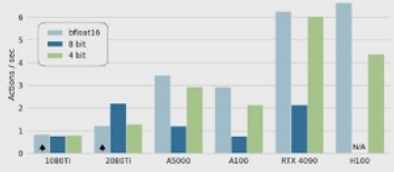


Figure 6: **OpenVLA inference speed for various GPUs.** Both bfloat16 and int4 quantization achieve high throughput, especially on GPUs with Ada Lovelace architecture (RTX 4090, H100). Further speed-ups are possible with modern LLM inference frameworks like TensorRT-LLM [89]. ♠: Model sharded across two GPUs to fit.

Precision	Bridge Success	VRAM
bfloat16	71.3 ± 4.8%	16.8 GB
int8	58.1 ± 5.1%	10.2 GB
int4	71.9 ± 4.7%	7.0 GB

Table 2: **Performance with quantized inference.** 4-bit quantization matches the performance of bfloat16 inference (our default approach) while reducing the GPU memory footprint by more than half. Mean success ± StdErr computed across 8 representative BridgeData V2 tasks [6] and 80 rollouts per approach (see Table 5 for details).

OpenVLA, a 7B-parameter model, consumes more memory at inference time than prior open-source generalist policies such as Octo, which has <100M parameters. We follow best-practices from LLM serving by saving and loading OpenVLA in `bfloat16` precision for inference (our default approach), which cuts the memory footprint in half, allowing us to serve OpenVLA on GPUs with only 16GB

[4] In Section 5.3 and Section 5.4, we experiment with a version of the OpenVLA model that is pretrained with a smaller robot data mixture (the same OpenX dataset mixture as Octo) and has a slightly smaller architecture which only uses a SigLIP [79] vision backbone instead of the fused DinoSigLIP encoder. We find that this simpler architecture still achieves strong performance in both fine-tuning tasks and "out-of-the-box" tasks.

OpenVLA:
An Open-Source Vision-Language-Action Model

Moo Jin Kim*,1 Karl Pertsch*,1,2 Siddharth Karamcheti*,1,3
Ted Xiao[4] Ashwin Balakrishna[3] Suraj Nair[3] Rafael Rafailov[1] Ethan Foster[1] Grace Lam
Pannag Sanketi[4] Quan Vuong[5,†] Thomas Kollar[3] Benjamin Burchfiel[3] Russ Tedrake[3,6] Dorsa Sadigh[1]
Sergey Levine[2] Percy Liang[1] Chelsea Finn[1]

https://openvla.github.io

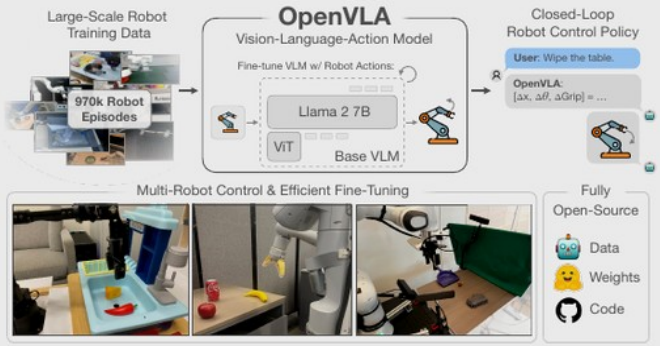


Figure 1: We present OpenVLA, a 7B-parameter open-source vision-language-action model (VLA), trained on 970k robot episodes from the Open X-Embodiment dataset [1]. OpenVLA sets a new state of the art for generalist robot manipulation policies. It supports controlling multiple robots out of the box and can be quickly adapted to new robot domains via parameter-efficient fine-tuning. The OpenVLA checkpoints and PyTorch training pipeline are fully open-source and models can be downloaded and fine-tuned from HuggingFace.

Abstract: Large policies pretrained on a combination of Internet-scale vision-language data and diverse robot demonstrations have the potential to change how we teach robots new skills: rather than training new behaviors from scratch, we can fine-tune such vision-language-action (VLA) models to obtain robust, generalizable policies for visuomotor control. Yet, widespread adoption of VLAs for robotics has been challenging as 1) existing VLAs are largely closed and inaccessible to the public, and 2) prior work fails to explore methods for efficiently fine-tuning VLAs for new tasks, a key component for adoption. Addressing these challenges, we introduce OpenVLA, a 7B-parameter open-source VLA trained on a diverse collection of 970k real-world robot demonstrations. OpenVLA builds on a Llama 2 language model combined with a visual encoder that fuses pretrained features from DINOv2 and SigLIP. As a product of the added data diversity and new model components, OpenVLA demonstrates strong results for generalist manipulation, outperforming closed models such as RT-2-X (55B) by 16.5% in absolute task success rate across 29 tasks and multiple robot embodiments, with 7x fewer parameters. We further show that we can effectively fine-tune OpenVLA for new settings, with especially

*: denotes equal contribution
Correspondence to: moojink@stanford.edu, pertsch@berkeley.edu, skaramcheti@stanford.edu
[1]Stanford University, [2]UC Berkeley, [3]Toyota Research Institute, [4]Google Deepmind, [5]Physical Intelligence, [6]MIT, [†]Work done in part while at Google Deepmind

Visionary · close

Harness: OpenVLA as bin-AR control. Deploy: $\pi_{0.5}$ / GR00T.

Steal LoRA recipe · FAST/chunks before high-Hz claims · no Discord/Drive

Q&A · Oct 21 · OpenVLA / 2406.09246

Harness vs deploy · Visionary → 2026 BEHAVIOR

Skip Discord / Drive entirely

HARNESS · keep

- OpenVLA as bin-AR control
- LoRA / int4 recipes
- Language multi-object evals
- OXE-style mix morals
- Demo cleaning (no-ops!)

DEPLOY · ship

- $\pi_{0.5}$ / OpenPI / GR00T-class
- Chunked continuous FM
- Optional FAST aux (not primary)
- Multi-cam + proprio + stages
- Task/stage tokens for fixed-50

1 Do not ship pure OpenVLA AR as primary 30 Hz controller

2 If AR research continues → FAST/chunks before high-Hz claims

3 Steal LoRA recipe into FM stacks

4 Honesty: Bridge/Google % ≠ BEHAVIOR q-score